

Comprehensive UI/UX Redesign

Specification and Architecture Blueprint

Strategic Context and Architectural Paradigm Shift

Mobile personal finance applications routinely fail because of visual and interaction fatigue caused by undifferentiated interface patterns¹. Traditional financial tracking applications present repetitive stacks of uniform white containers against muted backgrounds, creating an undifferentiated surface where high-priority alerts, discretionary allowances, and historical receipts compete for attention¹. This visual monotony obscures critical financial signals, requiring users to exert unnecessary cognitive effort simply to discern their daily liquidity¹. The Spending Tracker mobile application requires an architectural and visual overhaul to transition from a passive, retrospective ledger into a forward-looking cash flow intelligence system¹. The visual identity draws inspiration from contemporary tier-one consumer fintech platforms, specifically Revolut, Copilot Money, Apple Wallet, Linear, and Cash App¹. Rather than presenting numbers in isolation, the interface establishes an asymmetrical Bento Grid layout that groups related financial metrics into distinct functional tiles¹. Net liquidity, forward commitments, and category utilization are presented using an intentional surface hierarchy where luminance, hairline framing, and dynamic typography direct visual attention¹. The application operates as a privacy-focused, local-first system built on React Native using Expo SDK 57, Expo Router v4, and SQLite managed through Drizzle ORM¹. Financial tracking software that relies on cloud roundtrips introduces latency that discourages quick, on-the-go logging¹. By executing transactions on-device and using NativeWind v4 with React Native Reusables primitives, the application achieves near-zero interaction latency¹. Financial figures are displayed using tabular monospace typography (tabular-nums), ensuring that rapid account adjustments, real-time recalculations, and currency formatting never introduce horizontal layout shifts¹. This visual and ergonomic overhaul is achieved while preserving core domain algorithms, database schemas, and existing test contracts¹.

Dimension	Legacy Interface State	Redesigned Architectural Target
Visual Hierarchy	Symmetrical vertical stack of uniform white cards ¹	Asymmetrical Bento Grid with high-contrast hero metrics ¹
Color Spectrum	Off-white (#F7F8FA) with corporate green (#15803D) ¹	Obsidian Luxe (#090B10) with Electric Mint (#10B981) or Swiss Porcelain ¹

Depth & Elevation	Heavy drop shadows (elevation: 8) or flat borders (#E5E7EB) ¹	Hairline borders (border border-border/60) and tonal surface shifts ¹
Data Typography	Proportional system type causing horizontal jitter ¹	Fixed-width tabular monospace typography (tabular-nums) ¹
Controls & Density	Multi-row stacked filter chips crowding viewport ¹	Single-axis horizontal filter dock with sticky aggregate indicators ¹
Interactive Polish	Static touch boundaries lacking physical resonance ¹	Minimum 44 px to 48 px hit targets with tactile haptic feedback ¹

Deterministic Cash Flow Formulation and Mathematical Model

The user interface derives its hierarchy from an underlying cash flow calculation engine¹. Most personal finance trackers report historical aggregate spending, which forces the user to calculate their remaining budget manually¹. The Spending Tracker addresses this through a forward-looking Safe-to-Spend engine, calculating real-time discretionary spending limits by deducting unpaid fixed liabilities and monthly budget reserves from liquid capital¹.

To eliminate IEEE 754 floating-point rounding errors across platforms, all currency values are stored, calculated, and communicated as integer sen, where

$$100 \text{ sen} = 1.00 \text{ Malaysian Ringgit (RM)} \quad 1.$$

Cash Flow Engine Formulation

Liquid capital, denoted as B_{avail} , represents total unencumbered liquidity across checking, savings, and physical cash accounts, minus revolving debt obligations¹:

$$B_{\text{avail}} = \sum_{a \in A_{\text{liquid}}} \text{Balance}(a) - \sum_{c \in A_{\text{credit}}} \text{Debt}(c)$$

Forward liabilities for the current monthly cycle combine two values: upcoming unpaid fixed commitments (C_{unpaid}) and the remaining unspent budget reserve (R_{budget})¹. Unpaid commitments include loans, utility bills, and subscriptions that mature between the current timestamp t and the end of the monthly billing period T_{end} ¹:

$$C_{\text{unpaid}} = \sum_{i \in \text{Commitments}_{\text{active}}} \text{Amount}(i) \cdot \mathbb{I}(\text{DueDate}(i) \leq T_{\text{end}} \wedge \neg \text{IsPaid}(i))$$

The budget reserve prevents discretionary spending from exceeding the overall monthly spending limit¹. For an overall monthly target budget M_{target} and actual month-to-date spending S_{spent} , this reserve is calculated as¹:

$$R_{\text{budget}} = \max(0, M_{\text{target}} - S_{\text{spent}})$$

Discretionary liquidity (S_{safe}) and the derived daily spending allowance (A_{daily}) depend on the remaining days in the active month (D_{rem})¹:

$$S_{\text{safe}} = B_{\text{avail}} - C_{\text{unpaid}} - R_{\text{budget}}$$

$$A_{\text{daily}} = \frac{S_{\text{safe}}}{\max(1, D_{\text{rem}})}$$

The daily discretionary allowance serves as the primary visual metric on the dashboard¹. When actual expenses increase mid-month, R_{budget} decreases concurrently, freeing up discretionary capital if the user remains under their overall spending target¹. Conversely, if spending outpaces the target, the Safe-to-Spend pool contracts to safeguard upcoming bill payments¹.

Health Classification	Mathematical Boundary	Visual Surface Treatment	Status Pill Label
Healthy Surplus	$S_{\text{safe}} > 0 \wedge A_{\text{daily}}$ [cite: 1]	Obsidian surface with Electric Mint hairline perimeter (#10B981) ¹	HEALTHY SURPLUS [cite: 1]
Constrained Margin	$S_{\text{safe}} > 0 \wedge A_{\text{daily}}$ [cite: 1]	Warm Amber hairline perimeter (#F59E0B) with amber tint ¹	TIGHT MARGIN [cite: 1]

Deficit / Shortfall	$S_{\text{safe}} \leq$ [cite: 1]	Vivid Rose hairline perimeter (#F43F5E) with crimson wash ¹	DEFICIT WARNING [cite: 1]
---------------------	-------------------------------------	---	------------------------------

Design Tokens, Surface Optics, and Typography System

The redesigned interface uses high-contrast surfaces to present numerical data clearly across varying lighting conditions¹. The core palette, **Obsidian & Electric Mint**, is optimized for mobile OLED screens, reducing power consumption while increasing visual contrast¹. The design also includes an inverted **Porcelain Swiss** theme for clean light-mode rendering¹.

The interface avoids artificial drop shadows, which can create visual clutter on high-density mobile screens¹. Depth is established instead through tonal separation and hairline borders¹. Elevated containers use subtle border framing (border border-border/60), matching the crisp presentation seen in developer tools and premium financial applications¹.

```
JavaScript
// tailwind.config.js
module.exports = {
  darkMode: 'class',
  content: ['./src/**/*.ts,tsx'],
  presets: require('nativewind/preset'),
  theme: {
    extend: {
      colors: {
        background: 'var(--background)',
        card: {
          DEFAULT: 'var(--card)',
          foreground: 'var(--card-foreground)'
        },
        popover: {
          DEFAULT: 'var(--popover)',
          foreground: 'var(--popover-foreground)'
        },
        border: {
          DEFAULT: 'var(--border)',
          subtle: 'var(--border-subtle)'
        }
      }
    }
  }
}
```

```
input 'var(--input)'
primary
  DEFAULT 'var(--primary)'
  foreground 'var(--primary-foreground)'
  background 'var(--primary-background)'
secondary
  DEFAULT 'var(--secondary)'
  foreground 'var(--secondary-foreground)'
  background 'var(--secondary-background)'
destructive
  DEFAULT 'var(--destructive)'
  foreground 'var(--destructive-foreground)'
  background 'var(--destructive-background)'
warning
  DEFAULT 'var(--warning)'
  foreground 'var(--warning-foreground)'
  background 'var(--warning-background)'
muted
  DEFAULT 'var(--muted)'
  foreground 'var(--muted-foreground)'
  background 'var(--muted-background)'
accent
  DEFAULT 'var(--accent)'
  foreground 'var(--accent-foreground)'
  background 'var(--accent-background)'
fontFamily
  mono 'SpaceMono' 'Courier New' 'monospace'
  sans 'Inter' 'system-ui' 'sans-serif'
```

Design Token	Obsidian Palette (Dark)	Porcelain Palette (Light)	Architectural Function
--background	#090B10 [cite: 1]	#F8FAFC [cite: 1]	Canvas foundation that establishes maximum contrast ¹

--card	#121620 [cite: 1]	#FFFFFF [cite: 1]	Primary Bento Grid tile surface ¹
--card-foreground	#F8FAFC [cite: 1]	#0F172A [cite: 1]	High-contrast text and numerical values ¹
--border	#21293A [cite: 1]	#E2E8F0 [cite: 1]	Structural hairline framing (0.5 pt equivalent) ¹
--border-subtle	#18202F [cite: 1]	#F1F5F9 [cite: 1]	Internal dividers between related metrics ¹
--input	#1D2433 [cite: 1]	#F1F5F9 [cite: 1]	Background surface for text entry and keypad targets ¹
--primary	#10B981 [cite: 1]	#059669 [cite: 1]	Electric Mint; indicates healthy cash flow and positive balances ¹
--primary-foreground	#022C22 [cite: 1]	#FFFFFF [cite: 1]	Contrast text for primary buttons and chips ¹
--destructive	#F43F5E [cite: 1]	#E11D48 [cite: 1]	Vivid Rose; highlights deficits and budget overruns ¹
--warning	#F59E0B [cite: 1]	#D97706 [cite: 1]	Warm Amber; denotes approaching bills and budget limits ¹
--muted	#1A2130 [cite: 1]	#F1F5F9 [cite: 1]	Recessed backgrounds for chips and meter

			tracks ¹
--muted-foreground	#7C8BA1 [cite: 1]	#64748B [cite: 1]	Secondary labels and timestamp indicators ¹
--accent	#06B6D4 [cite: 1]	#0891B2 [cite: 1]	Cyber Cyan; marks linked commitments and AI evaluations ¹

Typography follows a clear scale tailored for numerical readability¹. Primary amounts use a bold, tightly tracked display font (text-5xl font-black font-mono tracking-tighter) set with fontVariant: ['tabular-nums']¹. Card titles use an uppercase style (text-[11px] font-bold uppercase tracking-wider text-muted-foreground), and list item descriptions are styled with standard body text (text-base font-semibold text-foreground) paired with muted metadata captions (text-xs font-medium text-muted-foreground)¹.

Reusable Component Primitives and Design System Architecture

To avoid generic interface patterns, all screens are composed of reusable atomic primitives located in @/components/ui/¹. These components wrap React Native Reusables (RNR) and rn-primitives, ensuring accessibility compliance, ARIA role mapping, and responsive touch boundaries¹.

The primitive library includes:

- MoneyDisplay.tsx: Sen-accurate tabular numeric renderer with Malaysian Ringgit formatting¹.
- StatusPill.tsx: Micro-badge container for financial health and cycle indicators¹.
- BudgetMeter.tsx: Multi-stage dynamic progress indicator that shifts color across budget thresholds¹.
- BentoCard.tsx: Modular surface container featuring hairline borders and consistent elevation¹.
- TouchTarget.tsx: Interactive wrapper enforcing minimum 44 px to 48 px touch boundaries¹.
- button.tsx, card.tsx, input.tsx, sheet.tsx: Core RNR layout and input components styled with hairline borders¹.

TypeScript

```

/ src/components/ui/MoneyDisplay.tsx
import React from 'react'
import { Text, TextProps } from 'react-native'

interface MoneyDisplayProps extends TextProps {
  amountInSen: number
  size?: 'xs' | 'sm' | 'base' | 'lg' | 'xl' | 'hero'
  trendPrefix?: boolean
  dimCents?: boolean
}

export function MoneyDisplay({
  amountInSen,
  size = 'base',
  trendPrefix = false,
  dimCents = false,
  className = '',
  ...props
}: MoneyDisplayProps) {
  const isNegative = amountInSen < 0
  const absSen = Math.abs(amountInSen)
  const ringgitPart = Math.floor(absSen / 100).toLocaleString('en-MY')
  const senPart = (absSen % 100).toString().padStart(2, '0')

  const sizeClasses: Record = {
    xs: 'text-xs font-semibold',
    sm: 'text-sm font-semibold',
    base: 'text-base font-semibold',
    lg: 'text-lg font-bold tracking-tight',
    xl: 'text-2xl font-black tracking-tight',
    hero: 'text-4xl sm:text-5xl font-black tracking-tighter'
  }

  const prefix = trendPrefix ? (isNegative ? '-' : '+') : (isNegative ? '-' : '')

  return (
    

{prefix}{ringgitPart}
      {dimCents ? (
        {senPart}
      ) : (
        {senPart}
      )}


  )
}

```


TypeScript

```
// src/components/ui/StatusPill.tsx
```

```
import React from 'react'
```

```
import { View, Text } from 'react-native'
```

```
export type StatusVariant = 'healthy' | 'warning' | 'danger' | 'neutral' | 'accent'
```

```
interface StatusPillProps
```

```
  variant: StatusVariant
```

```
  label: string
```

```
  dot?: boolean
```

```
  className?: string
```

```
{
```

```
export function StatusPill({ variant, label, dot = true, className = '' }: StatusPillProps) {
```

```
  const styles: Record =
```

```
    healthy: {
```

```
      bg: 'bg-emerald-500/10 dark:bg-emerald-500/15 border-emerald-500/30'
```

```
      text: 'text-emerald-700 dark:text-emerald-400'
```

```
      dot: 'bg-emerald-500'
```

```
    },
```

```
    warning: {
```

```
      bg: 'bg-amber-500/10 dark:bg-amber-500/15 border-amber-500/30'
```

```
      text: 'text-amber-700 dark:text-amber-400'
```

```
      dot: 'bg-amber-500'
```

```
    },
```

```
    danger: {
```

```
      bg: 'bg-rose-500/10 dark:bg-rose-500/15 border-rose-500/30'
```

```
      text: 'text-rose-700 dark:text-rose-400'
```

```
      dot: 'bg-rose-500'
```

```
    },
```

```
    neutral: {
```

```
      bg: 'bg-slate-500/10 dark:bg-slate-500/15 border-slate-500/30'
```

```
      text: 'text-slate-700 dark:text-slate-400'
```

```

    dot: 'bg-slate-400'
  },
  accent: {
    bg: 'bg-cyan-500/10 dark:bg-cyan-500/15 border-cyan-500/30',
    text: 'text-cyan-700 dark:text-cyan-400'
  },
  dot: 'bg-cyan-500'
},
],
],
],

const current = styles[variant],

return (

  <div id={id} &#x2F;>

    {label}

  </div>

)

```

TypeScript

```

// src/components/ui/BudgetMeter.ts
import React from 'react'
import { View } from 'react-native'

interface BudgetMeterProps {
  spentSen: number
  totalSen: number
  customColor?: string
  heightClass?: string
}

export function BudgetMeter({
  spentSen,
  totalSen,
  customColor,
  heightClass = 'h-2'

```

```

    { BudgetMeterProps }

    const percentage = totalSen > 0 ? Math.min(Math.round((spentSen / totalSen) * 100),
    100) : 0;

    const isOverBudget = spentSen > totalSen && totalSen > 0;

    const defaultFillColor = isOverBudget
    ? 'bg-destructive'
    : percentage >= 80
    ? 'bg-warning'
    : 'bg-primary';

    return (
      <View style={styles.container}>
        <Text>

```

TypeScript

```

// src/components/ui/BentoCard.ts
import React from 'react';
import { View, ViewProps } from 'react-native';

interface BentoCardProps extends ViewProps {
  elevated?: boolean;
  className?: string;
}

export function BentoCard({ children, elevated = false, className = '', ...props }:
BentoCardProps) {
  return (
    <View style={styles.container} className={className} {
      ...props
    }>
      {children}
    </View>
  );
}

```

```

TypeScript
src/components/ui/TouchTarget.tsx
import React from 'react'
import { Pressable, PressableProps } from 'react-native'

interface TouchTargetProps extends PressableProps {
  minHeight?: number
  className?: string
}

export function TouchTarget({
  children,
  minHeight = 42,
  className = '',
  style,
  ...props
}: TouchTargetProps) {
  return (
    

```

Screen-by-Screen Technical Specifications

The application layout uses five tab routes, one persistent creation modal, and dedicated stack screens for detail audits and edits¹.

The route architecture is organized as follows:

- (tabs)/index.tsx: Dashboard displaying the Safe-to-Spend hero card and upcoming bills¹.
- (tabs)/expenses.tsx: Transaction ledger with filters, search, and grouped date feeds¹.
- (tabs)/budgets.tsx: Budget allocation manager with overall and category progress meters¹.
- (tabs)/commitments.tsx: Commitments manager for subscriptions, loans, and one-off debts¹.
- (tabs)/analytics.tsx: Financial analytics screen with category share charts and BYOK AI reviews¹.
- (tabs)/settings.tsx: Settings screen managing accounts, payroll distribution, and AI keys¹.
- expenses/new.tsx: Fast modal entry using a POS-style integer keypad¹.

- expenses/[id]/index.tsx: Transaction detail audit view¹.
- expenses/[id]/edit.tsx: Transaction edit view enforcing linked commitment locks¹.
- commitments/new.tsx: Commitment provisioning modal¹.
- commitments/[id]/index.tsx: Commitment history and payment settlement audit view¹.
- commitments/[id]/edit.tsx: Commitment terms adjustment view¹.

Screen 1: Dashboard (src/app/(tabs)/index.tsx)

The dashboard serves as the central control hub, communicating current discretionary liquidity through an asymmetrical Bento Grid¹.

The primary element is the **Hero Bento Tile**, which presents the daily allowance (A_{daily}) and total discretionary pool (S_{safe})¹. This container uses an obsidian surface framed with an Electric Mint border (border-primary/40) in surplus conditions, switching to Amber or Rose if limits tighten or shortfalls occur¹. A status badge in the upper right displays the cash flow health state (HEALTHY SURPLUS, TIGHT MARGIN, or DEFICIT WARNING)¹.

Below the hero tile sits a **Modular Bento Split Row**:

- The left tile displays the **Cash Flow Equation**, showing liquid balances minus unpaid commitments and budget reserves¹. Tapping this tile opens an accessible bottom sheet detailing the underlying math¹.
- The right tile houses the **Upcoming Commitments Countdown**, showing the next two maturing bills, their due dates (e.g., in 3 days), and a quick-pay action button¹.

The lower section contains the **Monthly Budget Snapshot**, rendering spending progress through BudgetMeter.tsx, followed by the **Contextual AI Daily Insight Card**¹. This card displays spending observations powered by on-device BYOK prompts, such as comparisons to prior-month velocity¹. A floating action dock pinned above the tab bar provides one-tap triggers for + Add Expense, Transfer, and Scan Receipt¹.

Dashboard Visual Section	Architectural Component	Structural Layout & Style	Preserved Test Contract
Hero Discretionary Display	BentoCard + MoneyDisplay [cite: 1]	Hero typography, status badge, Electric Mint border ¹	testID="dashboard-screen" [cite: 1]
Cash Flow Breakdown	BentoCard + Sheet [cite: 1]	Tabular math: Liquid - Bills - Reserve = Safe ¹	testID="cashflow-formula-card" [cite: 1]
Commitments	BentoCard + TouchTarget	Dual-bill countdown chips with	testID="upcoming-bills-card"

Countdown	[cite: 1]	quick-settle trigger ¹	[cite: 1]
Monthly Budget Snapshot	BentoCard + BudgetMeter [cite: 1]	Spent vs target progress with remaining balance ¹	testID="budget-overall-card" [cite: 1]
Contextual AI Widget	BentoCard + Button [cite: 1]	Single-tap query button with bullet analysis ¹	testID="ai-daily-insight" [cite: 1]

Screen 2: Expenses Feed (src/app/(tabs)/expenses.tsx)

The expenses feed functions as an audit ledger designed for quick scanning and filtering¹. Dense, multi-row button controls are replaced with an integrated search bar and a single horizontal filter carousel¹. The filter dock includes date presets (Today, This Week, This Month, Last Month, Custom Date), category chips with icon glyphs, and an account filter dropdown¹. A sticky summary strip framed with a hairline border (border-b border-border/60) displays aggregate figures for the filtered period, combining total spend via MoneyDisplay with an active transaction count badge¹.

Transactions are presented within daily card containers rather than flat text lists¹. Each container features a date header with that day's total spend, with individual transaction rows divided by hairline borders¹:

- *Left:* A ^{40 px} circular category icon container with a tinted background¹.
 - *Center:* Primary payee/note label, accompanied by metadata (timestamp, category, and payment account)¹.
 - *Right:* Formatted deduction amount (-RM 24.50) rendered with MoneyDisplay¹.
- Auto-generated transactions feature an outline pill tag: Linked Commitment¹.

Tapping a transaction navigates to its details (/expenses/[id]), while swipe actions expose options for editing and deletion¹. A floating action button (+) remains accessible in the lower right corner¹.

Feed Section	Layout Pattern	Component Primitives	Test Identifiers
Search & Filter Dock	Sticky top bar with horizontal scroll ¹	Input, TouchTarget, Badge [cite: 1]	testID="expense-search-input" [cite: 1]
Period Summary Strip	Full-width status strip with hairline	MoneyDisplay, StatusPill	testID="period-summary-strip"

	border ¹	[cite: 1]	[cite: 1]
Daily Grouped Cards	Modular card containers per day ¹	BentoCard, Separator [cite: 1]	testID="expenses-list" [cite: 1]
Transaction Row	Three-column row with 48 px touch target ¹	TouchTarget, MoneyDisplay, Badge [cite: 1]	testID="expense-row-{id}" [cite: 1]

Screen 3: Budgets Manager (src/app/(tabs)/budgets.tsx)

The budgets screen manages overall monthly spending limits and per-category allocations¹. Navigation between past, present, and future cycles is handled via an accessible calendar stepper (< October 2026 >)¹.

The **Overall Monthly Budget Card** (testID="budget-overall-card") serves as the central visual element, featuring a multi-tier BudgetMeter.tsx that changes color as spending approaches the monthly limit¹:

- Spending under 80% shows as Electric Mint (#10B981)¹.
- Spending between 80% and 99% shifts to Warning Amber (#F59E0B)¹.
- Spending at or above 100% displays as Destructive Rose (#F43F5E), accompanied by an over-budget badge showing the excess amount¹.

The card also displays remaining funds, total spent, and the remaining daily buffer (

$$S_{\text{safe}} / D_{\text{rem}})^1.$$

Below the overall card sits a **Two-Column Category Allocations Grid** covering all 12 categories¹. Each tile displays its category icon, current spend versus target cap, and an individual progress meter¹. Unset categories display a simple Set limit trigger, while exceeded budgets show an alert badge (Over by RM 50.00)¹. Tapping a tile opens an entry sheet to adjust limits via a numeric keypad¹.

Category Element	Layout Pattern	Visual State	Preserved Test ID
Overall Monthly Card	Bento hero card with multi-stage meter ¹	Shifts color at 80% and 100% utilization ¹	testID="budget-overall-card" [cite: 1]
Category Limit Card	Asymmetric grid tile (48 px target) ¹	Category-specific color token and progress bar ¹	testID="category-card-{id}" [cite: 1]

Budget Limit Sheet	Native gesture-driven bottom sheet ¹	Sen keypad with quick percentage presets ¹	testID="category-limit-input" [cite: 1]
---------------------------	---	---	--

Screen 4: Commitments and Subscriptions (src/app/(tabs)/commitments.tsx)

The commitments manager tracks three distinct obligation models: amortizing fixed loans, recurring subscriptions, and one-off upcoming obligations¹.

Entries are styled as digital passes, providing clear due dates and payment actions¹:

- **Fixed Amortizing Loans:** Display total principal, monthly payment amount, and a progress bar showing loan payoff status, remaining balance, and months left to maturity¹.
- **Recurring Subscriptions:** Display the recurring billing amount, renewal schedule, and linked payment account¹.
- **One-Time Obligations:** Display the single maturity date and full amount due¹.

Tapping "Mark as Paid" (testID="mark-paid-button") initiates an automated ledger synchronization flow, opening an account selector sheet to confirm payment¹:

- It generates a linked expense entry in the database¹.
- It deducts the amount from the chosen funding account¹.
- It marks the commitment as Paid for the active billing cycle¹.
- If a payment is subsequently reversed or deleted, the linked expense is removed and the account balance is restored¹.

Commitment Type	Primary Visual Metrics	Settle Workflow	Contract Markers
Fixed Amortizing Loan	Repayment progress, remaining balance, payoff date ¹	Generates linked expense; auto-decrements balance ¹	testID="loan-card-{id}" [cite: 1]
Recurring Subscription	Monthly fee, due date countdown, auto-debit account ¹	Generates linked expense; marks cycle paid ¹	testID="subscription-card-{id}" [cite: 1]
One-Off Obligation	Single maturity date, total amount, settled badge ¹	Generates linked expense; archives obligation ¹	testID="obligation-card-{id}" [cite: 1]

Screen 5: Analytics and Insights (src/app/(tabs)/analytics.tsx)

The analytics interface turns transaction histories into actionable insights, pairing a Month-over-Month (MoM) stepper with spending metrics and Bring-Your-Own-Key (BYOK) AI

reviews¹.

A $2 \times$ metrics grid presents key monthly indicators¹:

- Total monthly spend with MoM percentage change (e.g., -8.4% vs last month)¹.
- Daily average spend, calculated as total spend divided by days elapsed¹.
- Highest single spending day, highlighting the date and primary driver¹.
- Top spending category, showing its total amount and percentage of overall spend¹.

Below the grid, a **Category Breakdown Container** presents categories ranked by expenditure, with proportion bars and percentage chips¹.

The screen concludes with the **AI Monthly Review Container** (testID="ai-analysis-card"), framed with a subtle cyan border (border-cyan-500/40)¹. Tapping "Analyze Month" sends a local, privacy-safe spending summary to the user's configured Gemini or DeepSeek API¹. The response is displayed as structured bullet points covering an executive summary, spending anomalies, and recommendations, supported by error handling for offline or invalid API key states¹.

Analytics Container	Presentation Pattern	Data Visualizer Component	Preserved Contract
Metrics Bento Matrix	$2 \times$ modular tile grid ¹	BentoCard + MoneyDisplay [cite: 1]	testID="analytics-kpi-grid" [cite: 1]
Category Breakdown	Ranked list with horizontal proportion bars ¹	Proportional dynamic fill bars ¹	testID="analytics-categories" [cite: 1]
AI Monthly Review	Elevated card with subtle cyan border glow ¹	Structured bullets with retry states ¹	testID="ai-analysis-card" [cite: 1]

Screen 6: Settings, Accounts, and Payroll (src/app/(tabs)/settings.tsx)

The settings screen manages account balances, monthly payroll distributions, and BYOK AI configurations¹.

Accounts are presented using payment card styling¹:

- Liquid accounts (cash, checking, savings) display positive balances via MoneyDisplay¹.
- Revolving credit card accounts display outstanding debt in red (OWED: - RM 1,200.00)¹.
- Tapping an account opens a balance adjustment modal that creates an audit entry for tracking adjustments¹.

The **Standing Payroll Allocation Engine** allows users to configure automated salary distribution across accounts (e.g., Maybank: RM 3,000, Touch 'n Go: RM 500, Savings: RM

1,500)¹. Tapping "Payroll In" (testID="trigger-payroll-btn") credits all configured accounts in a single transaction and logs the timestamp¹. The **AI Provider Manager** provides radio options to toggle between Google Gemini (Flash / Pro) and DeepSeek (Chat / Reasoner)¹. API keys are validated and stored locally in encrypted on-device storage, ensuring that personal financial data remains strictly local¹.

Settings Section	Interface Metaphor	Primary Actions	Preserved Test Contract
Connected Accounts	Stacked payment cards ¹	Add Account, manual balance correction ¹	testID="accounts-stack" [cite: 1]
Payroll Engine	Split visualizer with percentage pills ¹	Adjust allocations, trigger "Payroll In" ¹	testID="trigger-payroll-btn" [cite: 1]
AI Configuration	Radio model selector + secure input ¹	Validate API key, toggle provider ¹	testID="ai-config-section" [cite: 1]

Screen 7: Fast Expense Entry Modal (src/app/expenses/new.tsx)

The expense entry flow is designed for speed, allowing users to log an expense in under three seconds via a POS-style keypad¹. The entry flow centers around the **POS-Style Money Input** (testID="money-input"), where integer keystrokes format dynamically into sen (e.g., entering 1-2-5-0 formats immediately as RM 12.50)¹. This eliminates cursor jumping and floating-point errors¹. Below the amount display, an **Account Selector** previews balance deductions in real time (e.g., Maybank: RM 3,450.00 → RM 3,437.50), showing the immediate financial impact of the transaction¹. A 3-column category grid provides 48 px touch targets, automatically remembering the last-used category for recurring entries¹. Date selectors provide quick toggles for Today and Yesterday, alongside an optional text field for brief notes¹. If a user edits an expense that was auto-generated by a commitment (expenses/[id]/edit.tsx), manual modifications and deletion controls are locked¹. The screen displays an explanatory badge: "This expense is linked to an active recurring commitment. Manage or reverse this transaction from the Commitments tab"¹.

Fast Entry Component	Layout Pattern	Interaction Physics	Contract Marker
----------------------	----------------	---------------------	-----------------

POS Amount Display	Centered hero typography (text-5xl font-black) ¹	Direct integer sen mapping; fixed RM prefix ¹	testID="money-input" [cite: 1]
Projected Balance Pill	Interactive horizontal chips ¹	Real-time balance deduction preview ¹	testID="account-pill-{id}" [cite: 1]
Category Matrix	3-column grid with 48 px targets ¹	Tactile haptic feedback on selection ¹	testID="category-matrix" [cite: 1]
Commitment Lock Badge	Outline warning badge with lock icon ¹	Disables editing; links to Commitments tab ¹	testID="linked-commitment-lock" [cite: 1]

Mobile Ergonomics, Viewport Engineering, and Keyboard Dynamics

Mobile financial applications require careful viewport handling to remain responsive across varying screen sizes, hardware cutouts, and on-screen keyboards¹.

Safe Area Insets and Layout Offsets

The application uses react-native-safe-area-context to handle device-specific safe areas dynamically¹. Fixed viewport height units (100vh) are avoided, as mobile system bars dynamically alter the active visible area¹.

Top navigation headers pad content based on insets.top to clear camera cutouts and status bars, while bottom navigation docks apply padding using insets.bottom to stay clear of system

home indicators¹. A standard horizontal gutter of 16 px (px-4) is maintained across all screens to ensure visual alignment and touch comfort¹.

The viewport layers are arranged as follows:

- Dynamic status bar and sensor cutout padding (insets.top)¹.
- Header and navigation controllers¹.
- Scrollable content container managed via KeyboardAwareScrollView¹.
- Interactive input fields dynamically positioned above the keyboard¹.
- Software keyboard surface¹.
- System home indicator padding (insets.bottom)¹.

Native Sheet Architecture and Keyboard Avoidance

Forms and modal entries use native gesture-driven bottom sheets built with rn-primitives and KeyboardAwareScrollView to prevent input fields from being obscured during text entry¹. The

scroll container automatically adjusts scroll offsets when the keyboard appears, keeping the active input field centered in the visible viewport¹.

Tactile Haptic Feedback Mapping

Tactile feedback is paired with financial interactions to provide physical confirmation of user actions, increasing confidence during data entry and settlements³.

User Trigger	Haptic Feedback Level	Behavioral Function
Keypad Digit Press	SelectionAsync (Light) ³	Replicates the feel of a physical POS terminal ¹
Category Selection	ImpactFeedbackStyle.Light [cite: 3]	Confirms category selection ¹
Commitment Mark as Paid	NotificationFeedbackType.Success [cite: 3]	Provides positive confirmation of debt settlement ¹
Budget Limit Reached	NotificationFeedbackType.Warning [cite: 3]	Alerts user when crossing budget thresholds ¹
Transaction Deletion	ImpactFeedbackStyle.Medium [cite: 3]	Confirms transaction removal ¹

Invariant Domain Boundaries, Data Contracts, and Verification Matrix

The visual redesign must strictly preserve all underlying business logic, database relationships, and automated test contracts¹.

Domain Engine Immutability

The mathematical calculation engine located in src/engine/ is production-verified and strictly immutable¹. The redesign must not alter any computational logic within the following core engine modules¹:

- src/engine/budgets.ts: Monthly budget caps, category allocation ratios, and threshold indicators¹.
- src/engine/cashflow.ts: Safe-to-Spend algorithms and daily allowance formulas¹.
- src/engine/commitments.ts: Amortization schedules, cycle status evaluation, and payment settlement logic¹.
- src/engine/totals.ts: Sen-level summation formulas and integer conversions¹.

- `src/engine/boundaries.ts`: Date boundaries, leap-year calculations, and month-end offsets¹.

Database Schema Invariants

All SQLite database models and Drizzle ORM schemas must remain unchanged¹. Financial figures must always be stored as integer `sen`¹.

Database Entity	Primary Key	Foreign Keys	Integer Invariant	Semantic Constraint
accounts	id (UUID) ¹	None ¹	balance_sen (Int) ¹	Stores balances in integer <code>sen</code> ; <code>is_credit</code> indicates revolving debt ¹
expenses	id (UUID) ¹	account_id, category_id, linked_commitment_id (nullable) ¹	amount_sen (Int) ¹	Tracks expenses in integer <code>sen</code> ; locks edits if commitment linked ¹
commitments	id (UUID) ¹	account_id (default funding) ¹	amount_sen, principal_sen, remaining_sen [cite: 1]	Tracks debt amortizations and recurring bills in integer <code>sen</code> ¹
budgets	id (UUID) ¹	category_id (nullable for overall) ¹	limit_sen (Int) ¹	Sets monthly overall and category spending limits in integer <code>sen</code> ¹

Complete Test Contract Preservation Matrix

To ensure the test suite continues to pass without regressions, all existing testID markers and accessibility attributes must be preserved across the redesigned components¹.

Screen Target	Preserved testID	Functional Verification
---------------	------------------	-------------------------

		Contract
Dashboard Screen	testID="dashboard-screen" [cite: 1]	Renders hero Safe-to-Spend figure and daily allowance ¹
Cash Flow Equation	testID="cashflow-formula-card" [cite: 1]	Displays liquid cash, unpaid bills, and budget reserves ¹
Upcoming Commitments	testID="upcoming-bills-card" [cite: 1]	Renders upcoming obligations and quick-settle trigger ¹
Budget Overall Card	testID="budget-overall-card" [cite: 1]	Validates monthly spent vs target metrics and status ¹
AI Context Insight	testID="ai-daily-insight" [cite: 1]	Validates context-aware spending velocity insight ¹
Search Input	testID="expense-search-input" [cite: 1]	Filters transaction rows by payee and remarks in real time ¹
Period Summary Strip	testID="period-summary-strip" [cite: 1]	Validates aggregate period totals and transaction count ¹
Expenses List Feed	testID="expenses-list" [cite: 1]	Validates date groupings, search, and category filtering ¹
Category Limit Field	testID="category-limit-input" [cite: 1]	Confirms budget cap updates for individual categories ¹
Loan Card Display	testID="loan-card-{id}" [cite: 1]	Confirms loan amortization progress and balance display ¹
Mark as Paid Action	testID="mark-paid-button"	Tests linked expense

	[cite: 1]	generation and balance deduction ¹
AI Analysis Review	testID="ai-analysis-card" [cite: 1]	Validates structured executive summary output ¹
Accounts Stack	testID="accounts-stack" [cite: 1]	Confirms account balance rendering across all account types ¹
Payroll Action Button	testID="trigger-payroll-btn" [cite: 1]	Executes multi-account salary distribution in one step ¹
POS Amount Input	testID="money-input" [cite: 1]	Verifies integer sen entry and currency formatting ¹
Category Matrix Grid	testID="category-matrix" [cite: 1]	Confirms category selection and keyboard avoidance ¹
Commitment Lock Tag	testID="linked-commitment-lock" [cite: 1]	Verifies read-only lock on expenses linked to commitments ¹

Phased Implementation Roadmap for Autonomous Agents

Autonomous agents implementing this redesign must proceed through four sequential phases¹. Each phase must be verified before proceeding to prevent cascading regressions¹.

Phase	Core Objective	Key Deliverables & Actions	Verification Criteria
Phase 1: Token Setup	Establish foundational tokens ¹	Update tailwind.config.js, global.css, colors.ts with Obsidian/Porcelain tokens ¹	CSS variables compile; type check clean ¹
Phase 2: Primitives	Build reusable UI primitives ¹	Implement MoneyDisplay, StatusPill,	Primitives meet 44 px to 48 px

		BudgetMeter, BentoCard, TouchTarget [cite: 1]	target boundaries ¹
Phase 3: Screen Overhaul	Refactor views progressively ¹	Update Dashboard, Feed, Budgets, Commitments, Analytics, Settings, Fast Entry ¹	All preserved testID markers confirmed present ¹
Phase 4: Verification	End-to-end quality assurance ¹	Run TypeScript validation, code linting, and automated test suite ¹	65 test suites and 800+ unit tests pass ¹

Architectural Directives

Implementing this blueprint transforms the Spending Tracker from a generic template into a high-performance personal finance application¹. Grounding the interface in an asymmetrical Bento Grid, tabular monospace typography, hairline surface framing, and tactile interactions creates an experience that matches tier-one fintech standards¹.

Adherence to these specifications guarantees that:

- Core calculations, integer sen arithmetic, and database schemas remain intact¹.
- Safe-to-Spend data is clearly communicated through structured visual states¹.
- The component library remains modular, accessible, and easily maintainable across future updates¹.
- Test suites and automation contracts continue to pass with complete fidelity¹.

Works cited

1. description.md
2. Your money, beautifully organized | Copilot Money – Personal, <https://www.copilot.money/>
3. Copilot Money Review: A Budgeting App that Finally Gets it Right, <https://moneywithkatie.com/copilot-review-a-budgeting-app-that-finally-gets-it-right/>
4. What is NativeWind and How to Use It: Tailwind CSS for React Native, <https://useaxentix.com/blog/nativewind/what-is-nativewind-and-how-to-use-it/>
5. Introduction - React Native Reusables, <https://reactnativereusables.com/docs>
6. Make It Animated - Nativewind, <https://www.nativewind.dev/resources>